

PRISM — Pull Request Intelligent Semantic Monitor

Tagline

A semantic risk analysis layer for pull requests.

Project Overview

PRISM is an AI-assisted pull request analysis system that detects semantic and behavioral risks before code is merged.

Traditional CI/CD systems mainly detect:

- syntax errors
- merge conflicts
- lint failures
- failing tests

But many production bugs happen even when all checks pass.

Examples:

- API contract drift
- enum/state inconsistencies
- downstream service assumptions breaking
- event payload mismatches
- authentication logic conflicts
- hidden dependency regressions

PRISM solves this by analyzing pull requests contextually instead of file-by-file.

The system builds a dependency graph around changed code, traces impacted components, and uses IBM Bob to reason about downstream behavioral risks.

Core Vision

Traditional PR systems answer:

“Can this code merge?”

PRISM answers:

“Should this code merge?”

Main Features

1. Pull Request Diff Analysis

PRISM analyzes pull request changes and extracts:

- modified files
- changed functions
- updated APIs
- enum changes
- state transition changes
- event payload modifications
- permission/authentication logic updates

The goal is to understand what actually changed beyond raw text differences.

2. Contextual Dependency Graph

Instead of treating files independently, PRISM constructs a localized repository graph around the changed code.

The graph may include:

- imports
- function calls
- API consumers/providers
- service dependencies
- event relationships
- database model relations
- route handlers

This allows the system to trace downstream impact.

Example:

```
PaymentService
  ↓ emits
RefundEvent
```

```
↓ consumed by  
AnalyticsService
```

If a payload or enum changes, PRISM identifies all impacted consumers.

3. Semantic Risk Detection

PRISM identifies logical and behavioral inconsistencies that traditional CI pipelines may miss.

Examples:

- state mismatch
- API contract drift
- broken downstream assumptions
- incompatible feature interactions
- permission logic conflicts
- missing state handling

Example:

```
status = "PENDING"
```

changed to:

```
status = "PROCESSING"
```

The code still compiles.

Tests may still pass.

But downstream analytics or notification systems may silently fail because they still expect `PENDING`.

PRISM detects these semantic inconsistencies.

4. IBM Bob Reasoning Engine

IBM Bob acts as the contextual reasoning layer of the system.

Bob receives:

- pull request diff
- surrounding code context
- impacted dependency graph
- affected services/components
- execution flow summaries

Bob is responsible for:

- understanding developer intent
- explaining semantic risks
- identifying suspicious downstream behavior
- summarizing architectural impact
- generating human-readable warnings
- generating targeted regression scenarios

Instead of developers manually tracing large repositories, Bob provides contextual interpretation of impact.

5. Automated Regression Test Generation

Based on impacted execution paths, PRISM generates targeted regression test suggestions.

Instead of generic tests, the system focuses on:

- changed state transitions
- impacted services
- modified contracts
- downstream execution flows
- affected edge cases

Example:

```
it('should handle PROCESSING state in analytics pipeline', () => {  
  expect(...)  
})
```

The goal is to reduce manual QA effort and catch hidden regressions earlier.

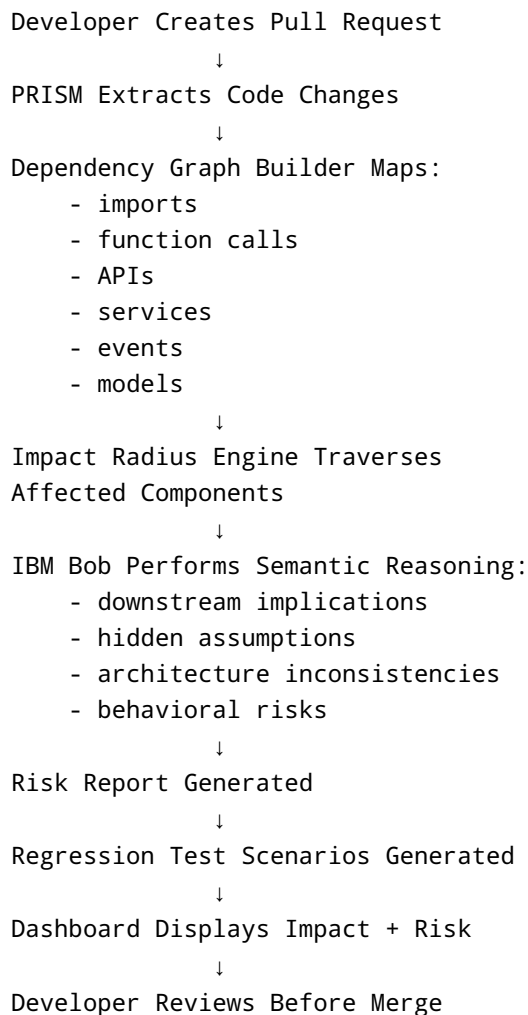
6. Risk Dashboard

PRISM provides a lightweight dashboard showing:

- semantic risk score
- impacted components
- dependency graph visualization
- IBM Bob reasoning summaries
- generated regression tests
- affected execution paths

The dashboard transforms PR review from reactive inspection into contextual risk analysis.

Full Workflow



Example Demo Scenario

A developer changes:

PENDING  PROCESSING

inside a payment service.

Traditional CI/CD:

- build passes
- tests pass
- merge succeeds

But another service still expects PENDING.

PRISM:

1. detects enum modification
2. traces impacted downstream services
3. identifies hidden assumptions
4. uses IBM Bob to explain semantic inconsistency
5. generates regression tests
6. highlights risk before merge

How Deep the System Goes

PRISM is not attempting full AGI-level code understanding.

The system focuses on:

- localized repository context
- dependency-aware reasoning
- impact tracing
- semantic inconsistency detection
- AI-assisted interpretation

It is intentionally scoped to feel:

- believable
- technically credible
- practically useful

- demoable in a hackathon
-

Surface-Level Tech Stack

Frontend

- Next.js
 - React
 - TailwindCSS
 - React Flow
-

Backend

- FastAPI
 - Python
-

Static Analysis

- tree-sitter
 - ts-morph
 - Python AST
-

Graph Engine

- NetworkX
 - Neo4j (optional)
-

AI Layer

- IBM Bob
-

IBM Bob Integration Strategy

IBM Bob is the semantic reasoning engine of PRISM.

The project is intentionally designed around Bob's advertised strengths:

- repository understanding
- code reasoning
- multi-file context awareness
- logic explanation
- workflow acceleration
- engineering intelligence

Bob is not merely used for code generation.

Instead, it is operationally integrated into the pull request analysis pipeline.

Bob is used for:

1. Contextual Repository Reasoning

Understanding relationships between changed code and downstream consumers.

2. Semantic Interpretation

Explaining how a logic change may alter system behavior.

3. Architectural Impact Summarization

Generating human-readable descriptions of affected services and execution flows.

4. Behavioral Risk Detection

Identifying suspicious assumption mismatches and semantic inconsistencies.

5. Regression Reasoning

Determining where hidden regressions are most likely to occur.

6. Targeted Test Generation

Generating contextual regression test scenarios based on impacted execution paths.

Final Positioning

PRISM is NOT:

- a generic AI code reviewer
- a Copilot clone
- a chatbot for developers

PRISM IS:

A semantic risk analysis layer for pull requests powered by contextual dependency reasoning and IBM Bob.